Securing the Agent: Vendor-Neutral, Multitenant Enterprise Retrieval and Tool Use

Francisco Javier Arceo
Red Hat AI
Boston, USA
farceo@redhat.com

Varsha Prasad Narsing
Red Hat AI
Boston, USA
vnarsing@redhat.com

Abstract

Retrieval-Augmented Generation (RAG) and agentic AI systems are increasingly prevalent in enterprise AI deployments. However, real enterprise environments introduce challenges largely absent from academic treatments and consumer-facing APIs: multiple tenants with heterogeneous data, strict access-control requirements, regulatory compliance, and cost pressures that demand shared infrastructure.

A fundamental problem underlies existing RAG architectures in these settings: retrieval systems rank documents by relevance—whether through semantic similarity, keyword matching, or hybrid approaches—not by authorization, so a query from one tenant can surface another tenant's confidential data simply because it scores highest. We formalize this gap and analyze additional shortcomings—including tool-mediated disclosure, context accumulation across turns, and client-side orchestration bypass—that arise when agentic systems conflate relevance with authorization. To address these challenges, we introduce a layered isolation architecture combining policy-aware ingestion, retrieval-time gating, and shared inference, enforced through server-side agentic orchestration. This approach centralizes security-critical operations—tool execution authorization, state isolation, and policy enforcement—on the server, creating natural enforcement points for multitenant isolation while allowing client-side frameworks to retain control over agent composition and latency-sensitive operations.

We validate the proposed architecture through an open-source implementation in OGX[1] [26], a vendor-neutral framework that implements an OpenAI-compatible, open-source Responses API with server-side multi-turn orchestration. We evaluate it empirically and show that ABAC gating eliminates cross-tenant leakage while introducing negligible overhead.

[1]OGX (Open GenAI Stack), formerly known as Llama Stack [19]. The project is available at https://github.com/ogx-ai/ogx; the Kubernetes Operator at https://github.com/ogx-ai/ogx-k8s-operator.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

CCS Concepts

• **Computing methodologies** → *Machine learning*; • **Information systems** → Data management systems.

Keywords

multitenancy, retrieval-augmented generation, access control, agentic AI, server-side orchestration, OGX, LLM systems, LLMOps

ACM Reference Format:
Francisco Javier Arceo and Varsha Prasad Narsing. 2026. Securing the Agent: Vendor-Neutral, Multitenant Enterprise Retrieval and Tool Use. In . ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Enterprise adoption of agentic AI introduces security challenges that existing architectures do not address. We motivate the problem, formalize it, and summarize our contributions.

1.1 Motivation

Enterprise adoption of generative AI has evolved beyond simple prompt-response interactions toward agentic systems—AI applications that autonomously reason, use tools, retrieve information, and execute multi-step workflows to accomplish complex tasks [13, 33, 39]. This evolution reflects a fundamental shift: rather than treating large language models (LLMs) as sophisticated text generators, organizations now deploy them as reasoning engines capable of taking actions in the world.

An API-first paradigm is emerging to unify multi-turn inference, tool use, and retrieval behind a single endpoint. OpenAI's Responses API is one prominent example, providing a unified interface for chat-style inference, tool invocation, retrieval, and stateful workflows [30], while open-source frameworks increasingly expose OpenAI-compatible endpoints to decouple applications from any single provider.

However, real-world enterprise deployments differ sharply from the assumptions embedded in consumer-facing APIs and many academic prototypes, exhibiting characteristics that demand specialized architectural consideration:

- **Multiple tenants:** distinct business units, customers, or partners served from shared infrastructure, with strict isolation requirements.
- **Heterogeneous data:** document collections vary in format, sensitivity classification, and access requirements.
- **Strict access control:** regulatory frameworks require fine-grained governance with auditable access patterns.

- **Operational control:** visibility into agent behavior, tool execution sequences, and data access patterns is required for debugging and compliance, consistent with lessons from production ML engineering [1].
- **Vendor independence:** lock-in to a single AI provider creates business risk; enterprises require on-prem and hybrid options.

Naïve approaches to addressing these requirements replicate the entire agentic stack per tenant: separate vector stores, dedicated inference endpoints, and isolated tool configurations. This strategy incurs substantial costs—infrastructure scales linearly with tenants rather than with actual usage—and creates operational fragmentation that amplifies common ML systems maintenance risks [34].

1.2 Problem statement

This paper addresses a fundamental tension in enterprise agentic AI deployment:

> Autonomous agents require flexible tool access and multi-turn reasoning capabilities, yet enterprise environments demand strict tenant isolation and policy enforcement—requirements that existing agentic architectures cannot simultaneously satisfy.

Standard agentic AI deployments exhibit security assumptions incompatible with enterprise multitenancy:

(1) **Client-side orchestration:** the application manages the inference → tool → inference loop, distributing security-critical logic to potentially untrusted clients and increasing operational complexity [1].
(2) **Homogeneous data access:** retrieval stacks assume uniform access to a corpus; retrieval methods (whether dense, sparse, or hybrid) optimize relevance ranking rather than authorization [14].
(3) **Implicit trust boundaries:** tool execution is often treated as a capability extension without systematic verification of who may invoke tools or consume tool outputs, despite the centrality of tool use in modern agent designs [13, 33, 39].
(4) **Stateless isolation:** requests are treated independently, ignoring how conversation state and cached tool results can leak across boundaries; such hidden couplings are a classic source of ML systems fragility [34].

In multitenant settings, these assumptions create serious vulnerabilities. A document highly similar to a query may belong to a different tenant. A tool call may access resources outside the user's authorization scope. Conversation history may accumulate context that crosses security boundaries.

This paper focuses on agentic RAG—the intersection of retrieval-augmented generation and autonomous tool use—as the primary case study, since retrieval is where the relevance-authorization gap is most acute. However, the layered isolation architecture and server-side enforcement patterns we propose are general and extend to other agentic capabilities including inference, tool execution, code generation, and multi-step workflows.

1.3 Contributions

This paper makes the following contributions:

(1) We formalize the *relevance-authorization gap* in multitenant RAG, showing why relevance ranking alone is insufficient to enforce isolation without explicit authorization predicates [12, 14], and provide empirical evidence that ungated retrieval leaks cross-tenant data in 98–100% of probes.
(2) We propose a layered isolation architecture combining policy-aware ingestion, retrieval-time gating, and shared inference, with server-side orchestration as the enforcement layer for the agentic control loop.
(3) We validate the architecture through a six-experiment evaluation measuring security (cross-tenant leakage, prompt injection resilience), systems performance (latency overhead, throughput scaling), and retrieval quality across a 2×2 configuration matrix crossing orchestration mode with retrieval gating.
(4) We implement the architecture in OGX [26], an open-source, vendor-neutral framework providing OpenAI-compatible APIs with pluggable providers for inference, vector stores, and tools, deployable on Kubernetes via a dedicated operator [25].

2 Background

We trace the evolution of LLM application architectures to show how each phase introduced new capabilities but also inherited the security gaps of prior phases.

2.1 The evolution of LLM application architectures

LLM application architectures have evolved through distinct phases, each introducing new capabilities and security considerations. **Completion APIs** exposed simple prompt → text interfaces with perimeter-oriented security.

Retrieval-augmented generation (RAG) [18] introduced retrieval and new attack surfaces such as retrieval manipulation and context poisoning, building on foundational work on prompt injection vulnerabilities [38]. **Dense retrieval and learned retrievers** (e.g., DPR [14] and retrieval-augmented pretraining such as REALM [10]) established the core technical basis for modern RAG. **Retrieval infrastructure** evolved to support multiple search modalities: dense vector search across diverse vector databases [12], sparse keyword matching (BM25), and hybrid approaches combining both with neural rerankers [32]. However, all of these modalities optimize for relevance; none enforce authorization natively. **Tool-using agents** [13, 33, 39] extended LLMs with tool calls and an inference → tool → inference loop. **Autonomous multi-step agents** execute multi-tool workflows with limited human oversight.

The Responses API paradigm [27, 30] represents a convergence of these phases, unifying inference, tool use, retrieval, and state management behind a single endpoint.

Each phase introduced new capabilities but also inherited the security gaps of prior phases. The Responses API paradigm unifies these capabilities under a single interface but assumes single-tenant deployment. Enterprise multitenancy requires policy-aware tool execution, stateful conversation management, and orchestration controls—challenges amplified in agentic deployments where reasoning, retrieval, and tool execution interleave autonomously [1, 34].

3 Architecture

We formalize the multitenant agentic AI environment: tenants (T) share infrastructure, each with associated data, users, tools, and policies. Agent execution follows the tool-using pattern [13, 39], where an execution sequence E consists of alternating inference steps i, tool calls ϕ, and responses r:

$$E = i_1, \phi_1, r_1, i_2, \phi_2, r_2, \ldots, i_n, \emptyset, r_n \tag{1}$$

where the final step has no tool call ($\emptyset$) and terminates with response r_n.

Standard agentic deployments exhibit several security assumptions that are fundamentally incompatible with enterprise multitenancy.

3.1 Security challenges

Relevance-authorization gap. Retrieval systems—whether vector-based, keyword-based, or hybrid—optimize for relevance metrics rather than authorization policies. This creates a fundamental gap: search ranking considers semantic similarity, term frequency, or combined relevance signals, but authorization decisions depend on access control policies that are orthogonal to these relevance measures.

For tenants T_A and T_B sharing corpus $D = D_A \cup D_B$, retrieval methods cannot enforce tenant isolation without an authorization predicate. Let q denote a query, u denote a user, d denote a document, θ denote a relevance threshold, and Pu, d denote an authorization policy that returns permit or deny. Then secure retrieval requires:

$$\{d \in D : \text{relevance}\big(q, d\big) > \theta \wedge P\big(u, d\big) = \text{permit}\} \tag{2}$$

Tool-mediated disclosure. Agents invoke tools with agent credentials rather than end-user authorization, potentially accessing unauthorized data across tenant boundaries.

Context accumulation. Multi-turn conversations persist context without per-turn policy re-validation, enabling cross-tenant data leakage.

Client-side bypass. When orchestration runs client-side, malicious clients can skip authorization checks, manipulate tool invocation, or extract unauthorized data.

These shortcomings stem from distributing security-critical logic outside the trust boundary [1, 34, 35].

3.2 Threat model and assumptions

We define the scope of adversaries and assumptions under which the architecture provides its security guarantees.

In-scope adversaries: (1) a *malicious tenant* crafting queries—including prompt injections—to retrieve another tenant's data; (2) a *compromised client* bypassing authorization by calling ungated endpoints or manipulating tool invocations; (3) a *buggy tool* leaking cross-tenant state through tool outputs or side effects.

Out-of-scope adversaries: a compromised server process, an insider operator with infrastructure access, side-channel attacks (timing, cache), and model extraction attacks. The trust boundary is the server process—all security-critical operations execute within it.

System goals:

- **G1** No cross-tenant data leakage through retrieval.
- **G2** Authorization enforcement independent of orchestration mode.
- **G3** Denied queries fail fast without invoking inference.

Assumptions: (A1) correct token-to-tenant mapping by the authentication provider; (A2) immutable document ownership metadata assigned at ingestion; (A3) the inference layer is untrusted—it may leak any context it receives, so isolation must be enforced before context construction; (A4) vector backends faithfully apply metadata filters when predicate pushdown is supported.

3.3 Data path: layered isolation

To address the security challenges, we propose a two-part solution. A three-layer isolation architecture secures the *data path*: how documents are ingested, retrieved, and fed to the model (Figure 1). Server-side orchestration secures the *control path*: how tools are invoked, state is managed across turns, and policies are enforced (Section 3.4).

Layer 1: Policy-aware ingestion. Tenant metadata attached at ingestion: $\mathcal{I}d, t \rightarrow D_t$ tags document d with tenant t's attributes. This ensures every chunk inherits ownership metadata, so downstream retrieval and authorization operate on consistent tenant attributes. Attaching metadata at ingestion rather than retrofitting it reduces the risk of accidental cross-tenant coupling [34].

Layer 2: Retrieval gating. Two-tier enforcement: resource-level ABAC authorization before search and chunk-level filtering after retrieval, composing similarity search with authorization predicates to implement Equation (2). Where the backend supports predicate pushdown, tenant filters are applied natively during vector search, maintaining perfect recall regardless of corpus size. On backends without pushdown, post-retrieval filtering preserves the security guarantee (G1) but recall degrades at large corpus sizes as cross-tenant documents contaminate the top-k set. We recommend pushdown-capable backends (e.g., pgvector, Qdrant, Milvus) for production deployments.

Layer 3: Shared inference. The LLM inference layer is shared across tenants; the model itself does not require per-tenant isolation, only the context fed to it. Because Layers 1 and 2 ensure that only authorized documents enter the prompt, the inference layer can be safely shared, reducing cost from $O(N \cdot M)$ to $O(M)$ where N is the number of tenants and M is the number of model endpoints.

This relies on assumption A3: the architecture secures *context construction*, not parametric memory. A model may generate information absorbed during pretraining regardless of retrieval gating. Per-tenant model instances can provide full parametric isolation but require deploying a separate model for each tenant, which is costly and impractical at scale. At the serving layer, modern systems show that batching, scheduling, and memory management dominate throughput and latency for generative transformers [15, 40].

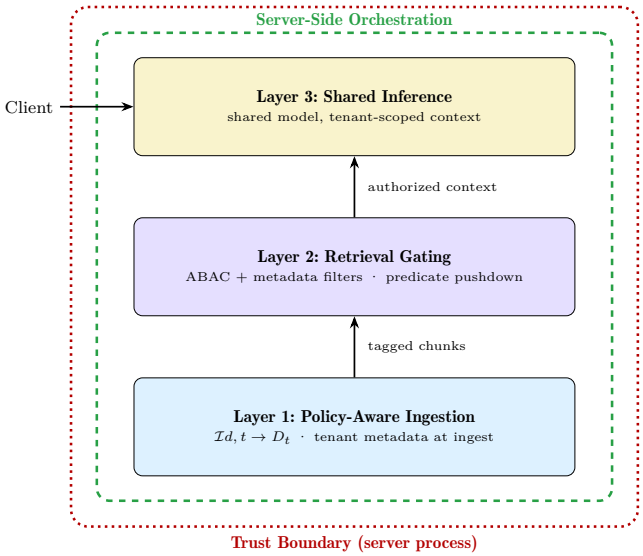

Figure 1: Layered isolation architecture with server-side orchestration. All three layers execute within the server trust boundary; the client controls *what* to ask but not *how* retrieval and tool execution are performed.

3.4 Control path: server-side orchestration

Client-side orchestration offers important advantages: low latency for local tool calls, flexibility in composing custom agent logic, and the ability to leverage rich client-side frameworks. However, in multitenant enterprise settings, purely client-side patterns expand the trusted computing base (TCB) to include untrusted client code. A compromised or buggy client can skip retrieval filters, invoke unauthorized tools, or accumulate cross-tenant context—the server cannot enforce security invariants it does not control [1, 34].

Server-side orchestration centralizes policy enforcement by executing the inference → tool → inference loop within the server trust boundary (Figure 2). This introduces trade-offs: added latency for tool calls that could execute locally and reduced flexibility for client-specific agent logic. For workloads where all tenants are trusted, client-side orchestration remains a pragmatic choice.

In practice, hybrid architectures are likely to emerge: server-side orchestration enforces security-critical invariants (authorization, state isolation, audit logging), while client-side frameworks retain control over agent composition and latency-sensitive operations.

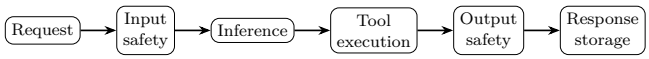

Figure 2: Server-side orchestration flow: every step runs inside the server trust boundary.

Table 1 maps each security challenge to the enforcement point that mitigates it.

Challenge	Enforcement point
Cross-tenant retrieval leakage	Layer 2 retrieval gating (ABAC + metadata filters)
Context accumulation	Tenant-scoped state storage and per-turn authorization
Tool-mediated disclosure	Server-side tool execution with authorization propagation
Client-side bypass	Server-side orchestration (reduced TCB)
Audit failure	Server-side telemetry and tracing

Table 1: Mapping from security challenges to architectural enforcement points.

4 Implementation

OGX [26] provides an open-source implementation of the architecture proposed in Section 3. The framework exposes OpenAI-compatible APIs for inference and agentic execution, enabling any client-side framework (LangChain [16], LangGraph [17], CrewAI [4], and others) to gain server-side policy enforcement, multitenancy, and provider portability without changes to agent code. Together with open models such as gpt-oss [29], this provides a complete open-source alternative to proprietary agentic AI platforms.

4.1 APIs and agentic execution

OGX defines over 20 APIs covering the full lifecycle of agentic applications. These span core capabilities—inference, agents (Responses API), vector stores and search, safety, tool runtime, and telemetry—as well as resource management APIs for models, files, file processors, prompts, conversations, connectors, and tool groups. The framework also provides compatibility layers for third-party API paradigms. Each API is designed with multitenancy as a first-class concern, enabling

tenant-scoped resource management and access control. A complete API listing is provided in Appendix B.

The Responses API—implementing the OpenAI Responses API paradigm [30]—is the central orchestration endpoint (Figure 5). Unlike chat completion APIs that terminate after a single inference call, a single Responses API request may trigger multiple inference calls, tool executions, safety checks, and state transitions before producing a final response. All such operations execute within the server boundary: conversation state is retrieved and persisted server-side, tools are invoked under centralized authorization, and safety guardrails are applied at each step.

The Vector Stores and Search APIs provide uniform access to multiple retrieval modalities—dense vector search, keyword matching, and hybrid pipelines—with structured metadata filtering for tenant isolation. The Prompts API enables versioned prompt management, and the Conversations API maintains tenant-scoped multi-turn state. Together, these APIs provide the OpenAI-compatible interface through which the architecture's security guarantees are delivered: clients interact with standard endpoints (`/v1/responses`, `/v1/vector_stores`, `/v1/chat/completions`) while the server enforces ABAC policies transparently.

4.2 Provider architecture

Extensibility is achieved through pluggable providers. Each API may be backed by multiple providers, categorized as *inline* (executing within the OGX process) or *remote* (adapting external services). This separation enables hybrid deployments: sensitive data paths remain local while computationally intensive operations are delegated to scalable external services. Crucially, provider substitution is transparent to clients—a developer can prototype with inline providers (e.g., sqlite-vec [8], an in-process safety model) and move to production-grade remote providers (e.g., pgvector, vLLM [15]) without changing application code or security policies.

A routing layer dispatches API requests to provider instances based on logical resource identifiers, incorporating authorization checks and tenant identity before delegation. Different tenants may be routed to distinct provider instances while sharing the same API surface. A *distribution* packages a specific set of APIs, providers, and resources into a deployable unit, decoupling application logic from provider selection. Supported providers are listed in Appendix B.

4.3 Access control

OGX includes a declarative, attribute-based access control (ABAC) framework that evaluates authorization at runtime. Access rules specify permit or deny scopes with conditions based on ownership and attribute matching. The default policy permits access when the user is the resource owner or when the user's attributes (roles, teams, projects, namespaces) match the resource's access attributes; a default-deny model ensures access is only granted when explicitly permitted.

Authorization is enforced at three levels: (1) API route middleware, (2) routing table resolution (before resolving a vector store or model to a provider), and (3) storage read time, where query filters are constructed from the current user's attributes so that tenants only see their own or attribute-matched rows. JWT or Kubernetes authentication providers map external identity claims into these attributes, so enterprise identity systems drive isolation without embedding tenant IDs in application logic.

4.4 Deployment

The OGX Kubernetes Operator [25] automates deployment through custom resources that declaratively specify server configurations, backend connections, and isolation policies. The operator supports shared instances (multiple tenants with ABAC isolation), per-tenant instances (namespace-level isolation with Kubernetes RBAC), and hybrid approaches [3]. In all topologies, the provider abstraction allows organizations to start with lightweight backends during development and migrate to production-grade infrastructure without modifying agent code or security policies.

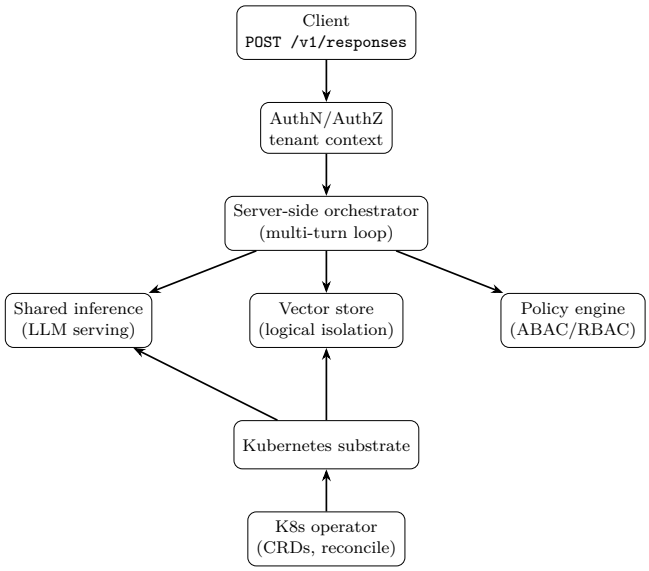

Figure 3: OGX architecture for multitenant enterprise agentic AI on shared Kubernetes infrastructure.

5 Evaluation

We evaluate the architecture through six experiments measuring security, systems performance, and retrieval quality. Figure 4 summarizes the key results. The evaluation uses a 2×2 configuration matrix crossing orchestration mode (client-side vs. server-side) against retrieval gating (ungated vs. ABAC-gated):

5.1 Setup

Workload. Three synthetic tenants (*finance*, *engineering*, *legal*) with 300 documents (100 per tenant, ∼512 tokens each)

	Ungated	Gated
Client-side	Config A	Config B
Server-side	Config C	Config D

Table 2: 2×2 configuration matrix.

and controlled topical overlap. Per configuration: 300 authorized queries (100 per tenant), 300 cross-tenant probes (a finance user querying for engineering documents), and 90 prompt injection probes spanning four attack categories (instruction override, role impersonation, debug exploitation, context manipulation).

Infrastructure. Inference via OpenAI `gpt-4o-mini` through OGX's `remote::openai` provider; embeddings via `text-embedding-3-small`; vector store via `inline::sqlite-vec`; authentication via a lightweight mock mapping bearer tokens to tenant identities. GPU infrastructure experiments use vLLM [15] serving `Llama-3.2-1B-Instruct` on an NVIDIA T4.

Metrics. Cross-Tenant Leakage Rate (CTLR): fraction of cross-tenant probes returning at least one unauthorized chunk. Authorization Violation Rate (AVR): fraction of all API calls returning unauthorized data.

5.2 Security results

The headline result: **gating is the security mechanism**. Without it, nearly all cross-tenant probes return unauthorized data regardless of orchestration mode. ABAC gating eliminates leakage entirely, confirming goal G1.

A natural question is why server-side orchestration matters if client-side gating (Config B) also achieves CTLR=0%. The answer is the trust boundary: Config B's security depends on the client faithfully calling the gated endpoint. A compromised client can skip gated search, call an ungated endpoint, or manipulate tool invocations—reverting to Config A's 100% leakage. Server-side orchestration (Config D) moves enforcement inside the server, where the client controls *what* to ask but not *how* retrieval and tool execution are performed. Gating provides the security guarantee; server-side orchestration provides the enforcement guarantee (G2).

Formal security claims. Under assumptions A1–A4 (Section 3.2), the architecture guarantees CTLR=0 and AVR=0 for any query workload: authorized queries return only permitted documents, and cross-tenant probes return no unauthorized data. The architecture does not protect against model prior knowledge leakage, side-channel attacks, or a compromised server process—these are explicitly out of scope.

Prompt injection resilience. Under gated configurations, all 90 injection probes achieve 0% leakage. The defense operates at the retrieval layer: the ABAC policy prevents the vector store from returning unauthorized documents regardless of prompt content. Under ungated configurations, 62–80% of probes retrieve cross-tenant data through normal relevance-based retrieval.

5.3 Performance results

Latency overhead. Isolating the search component from inference, the gated search path adds ∼19ms: auth server round-trip (∼14ms), ABAC policy evaluation (<1ms), and per-tenant store lookup (∼5ms). This is negligible relative to inference time (∼3–7s for API-based, ∼450ms for self-hosted GPU). On self-hosted GPU infrastructure (vLLM on T4), the OGX routing and dispatch layer adds 4.7ms (1.0% of baseline inference), and tenant metadata filtering adds 5.5ms (1.9% of search time). Full latency tables are in Appendix A.

Server-side orchestration adds ∼3s total latency vs. client-side due to the Responses API tool execution round-trip. All experiments used non-streaming responses (full completion before returning). With streaming—supported by OGX's Responses API—time-to-first-token would be substantially lower.

Throughput. Throughput scales linearly with concurrency up to c=25 across all configurations. Gating does not degrade throughput. Client-side orchestration achieves ∼2× the QPS of server-side at high concurrency due to the shorter request path (Appendix Table 5).

5.4 Retrieval quality and scaling

Controlled retrieval benchmarks. Using synthetic embeddings with ∼0.95 cross-tenant similarity to isolate the retrieval layer from API variance: ungated retrieval leaks 52% of queries. Chunk-level gating improves precision by 2.2× (Precision@5 from 0.200 to 0.433) and MRR from 0.700 to 1.000 by filtering cross-tenant noise. A 48-case ABAC correctness matrix achieves 100% accuracy with 0% false positives.

Predicate pushdown scaling. On backends without predicate pushdown (sqlite-vec), post-retrieval filtering maintains the security guarantee (CTLR=0%) but recall degrades at large corpus sizes: Recall@5 is 1.000 at 100 chunks but drops to 0.002 at 50K chunks. Filter latency overhead is small (0.7–3ms). Backends supporting predicate pushdown eliminate this trade-off entirely—we recommend pushdown-capable backends for production deployments (Appendix Table 7).

6 Discussion and Conclusion

We discuss the scope of the architecture, its trade-offs, and position relative to related work.

6.1 When not to use this architecture

This architecture is unnecessary for single-tenant deployments, public data, organizations already enforcing per-tenant infrastructure isolation, or teams that prefer a fully managed SaaS product over self-hosted infrastructure. Client-side orchestration achieves ∼2× the throughput at high concurrency for latency-sensitive workloads where all tenants are trusted. The framework is deployed in production across enterprises in telecommunications, semiconductor manufacturing, financial services, insurance, and consulting; the evaluation in this paper uses a synthetic testbed for reproducibility and controlled comparison.

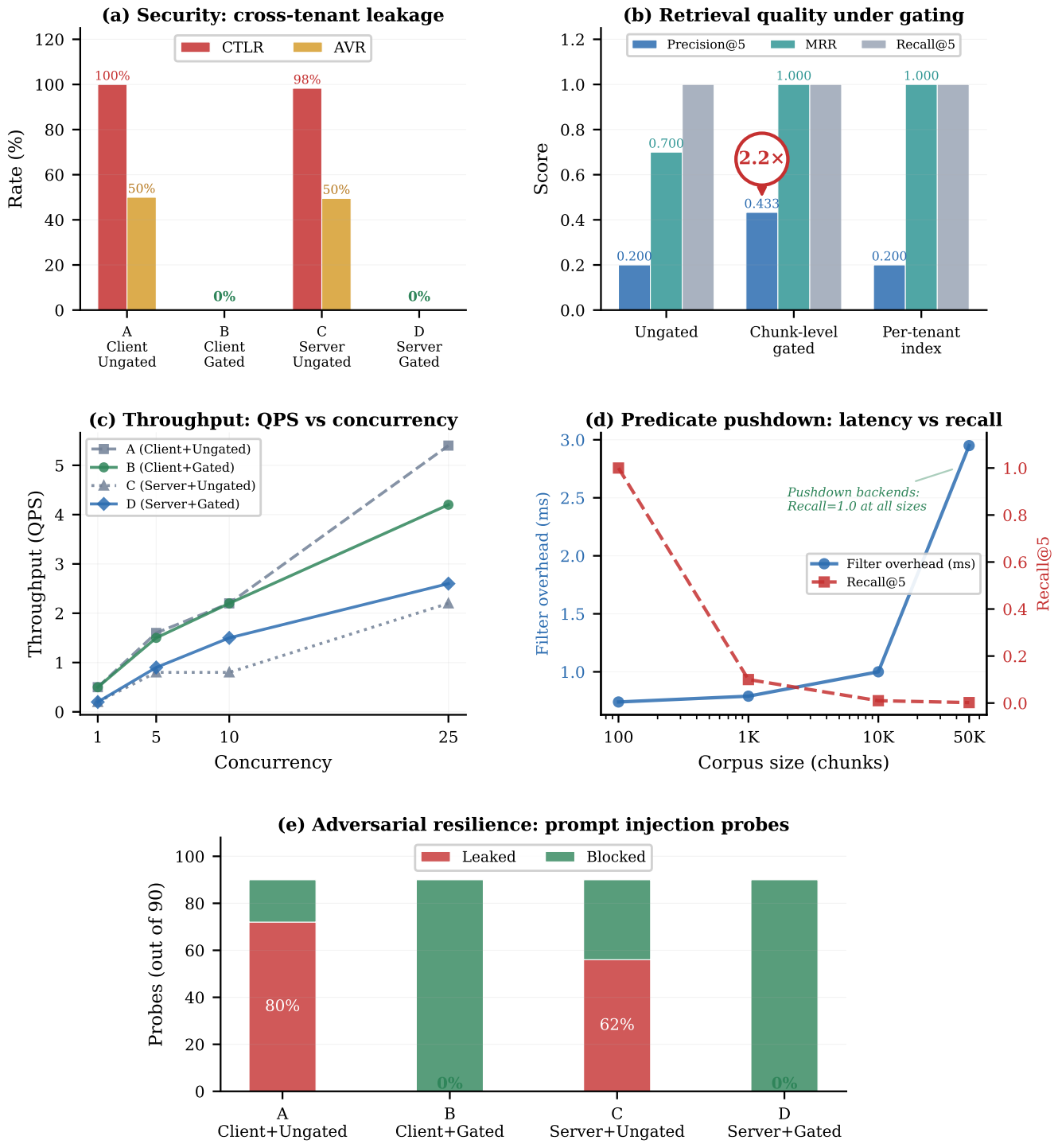

Figure 4: Empirical evaluation results across five dimensions. (a) Security: ABAC gating eliminates cross-tenant leakage (CTLR and AVR drop to 0%) regardless of orchestration mode. (b) Retrieval quality: chunk-level gating improves Precision@5 by 2.2× by filtering cross-tenant noise. (c) Throughput: gating does not degrade QPS; client-side achieves ∼2× server-side at high concurrency. (d) Predicate pushdown: post-retrieval filtering overhead is small but recall degrades at large corpus sizes; pushdown-capable backends maintain Recall@5=1.0. (e) Adversarial resilience: all 90 prompt injection probes blocked under gated configurations.

6.2 Design trade-offs

- **ABAC policy complexity.** Policy complexity grows with the number of tenants, roles, and resource types. Organizations with deeply nested permission structures may face policy management overhead.
- **Predicate pushdown.** As shown in Section 5, backends without native predicate pushdown experience recall degradation at scale. OGX enforces post-retrieval filtering across all backends regardless of native pushdown support, so the security guarantee holds; the trade-off is recall, not security.
- **Client-side function tools.** Client-side function tools, by design, execute outside the server trust boundary. OGX mitigates this through explicit tool classification but cannot enforce server-side invariants on client-executed code.

6.3 Related work

Several systems aim to standardize LLM-based agentic applications. The closest to our work are API-layer platforms and enterprise agent runtimes.

OpenAI's Responses API [30], function-calling conventions [28], and the Model Context Protocol (MCP) [2] establish *what* an agent can do but are silent on *who* may do it—they assume a single-tenant caller. Databricks' Mosaic AI Agent Framework [6] wraps agents in an MLflow ResponsesAgent [5, 41] with managed MCP tools and Unity Catalog governance, providing the closest enterprise parallel, but couples the experience to the Databricks stack. OGX adopts the same interface conventions while providing multitenant isolation through an open, vendor-neutral API layer deployable on any infrastructure.

LLM serving engines such as vLLM [15] and Orca [40] optimize inference throughput but are agnostic to tenant isolation; vector databases such as Milvus [42] and Weaviate [37] and platforms such as Vectara [36] rank by relevance without authorization enforcement. OGX treats these as pluggable providers beneath its authorization layer. Agent orchestration frameworks—LangGraph [17], Microsoft's Agent Framework [24] (successor to Semantic Kernel [23] and AutoGen [22]), Google ADK [9], Haystack [7], LlamaIndex [21], Smolagents [11], and Pydantic AI [31]—provide developer-facing abstractions but orchestrate from the client side; OGX serves as the server-side execution target these frameworks call into.

Production ML research has documented the fragility of distributed invariants [1, 34], and lifecycle platforms such as MLflow [41] address deployment but not agentic multitenancy. To our knowledge, no prior work addresses the intersection of standardized agentic API design, server-side orchestration, and multitenant isolation on shared infrastructure. The contribution is the composition of individually known techniques—ABAC, server-side orchestration, pluggable providers—for the specific problem of multitenant agentic AI, validated empirically.

6.4 Conclusion

Enterprise deployment of agentic AI systems introduces security and compliance challenges that existing architectures—designed for single-tenant, consumer-facing use—do not address. This paper formalized the relevance-authorization gap, proposed a layered isolation architecture with server-side enforcement, and evaluated it empirically: ABAC gating eliminates cross-tenant leakage entirely while adding ∼19ms to the search path, and throughput scales linearly with no gating bottleneck. The defense operates at the retrieval layer, making it resilient to prompt injection attacks regardless of model behavior.

Our implementation through OGX demonstrates that secure multitenancy, vendor-neutral OpenAI-compatible APIs, and autonomous agent capabilities are simultaneously achievable on shared infrastructure without per-tenant duplication.

References

[1] Saleema Amershi, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. 2019. Software Engineering for Machine Learning: A Case Study. In *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, Piscataway, NJ, USA, 291–300. doi:10.1109/ICSE-SEIP.2019.00042

[2] Anthropic. 2024. Model Context Protocol. Online documentation. https://modelcontextprotocol.io/docs/getting-started/intro Accessed: 2026-02-24.

[3] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. 2016. Borg, Omega, and Kubernetes. *Commun. ACM* 59, 5 (2016), 50–57. doi:10.1145/2890784

[4] CrewAI, Inc. 2024. CrewAI: Framework for orchestrating role-playing autonomous AI agents. Open-source project. https://github.com/crewAIInc/crewAI Accessed: 2026-02-23.

[5] Databricks. 2025. Author an agent in code using MLflow ResponsesAgent. Online documentation. https://docs.databricks.com/en/generative-ai/agent-framework/create-agent.html Accessed: 2026-02-24.

[6] Databricks. 2025. Mosaic AI Agent Framework. Online documentation. https://www.databricks.com/product/machine-learning/retrieval-augmented-generation Accessed: 2026-02-24.

[7] deepset. 2023. Haystack: End-to-end LLM framework for building production-ready applications. GitHub repository. https://github.com/deepset-ai/haystack Accessed: 2026-02-24.

[8] Alex Garcia. 2024. sqlite-vec: A vector search SQLite extension. GitHub repository. https://github.com/asg017/sqlite-vec Accessed: 2026-04-25.

[9] Google. 2025. Agent Development Kit (ADK). GitHub repository. https://github.com/google/adk-python Accessed: 2026-02-24.

[10] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: Retrieval-Augmented Language Model Pre-Training. In *International Conference on Learning Representations (ICLR)*. OpenReview, Addis Ababa, Ethiopia, 1–16. https://arxiv.org/abs/2002.08909

[11] Hugging Face. 2025. smolagents: A smol library to build great agents. GitHub repository. https://github.com/huggingface/smolagents Accessed: 2026-02-24.

[12] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2017. Billion-Scale Similarity Search with GPUs. *arXiv preprint arXiv:1702.08734* 1, 1 (2017), 1–17. https://arxiv.org/abs/1702.08734

[13] Ehud Karpas, Omri Abend, Yonatan Belinkov, Barak Lenz, Opher Lieber, Nir Ratner, Yoav Shoham, Hofit Bata, Yoav Levine, Kevin Leyton-Brown, Dor Muhlgay, Noam Rozen, Erez Schwartz, Gal Shachaf, Shai Shalev-Shwartz, Amnon Shashua, and Moshe Tenenholtz. 2022. MRKL Systems: A Modular, Neuro-Symbolic Architecture that Combines Large Language Models, External Knowledge Sources and Discrete Reasoning. *arXiv preprint arXiv:2205.00445* 1, 1 (2022), 1–24. https://arxiv.org/abs/2205.00445

[14] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih.

2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Online, 6769–6781. doi:10.18653/v1/2020.emnlp-main.550

[15] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*. ACM, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165

[16] LangChain, Inc. 2023. LangChain: Build context-aware reasoning applications. Open-source project. https://github.com/langchain-ai/langchain Accessed: 2026-02-23.

[17] LangChain, Inc. 2024. LangGraph: Build resilient language agents as graphs. Open-source project. https://github.com/langchain-ai/langgraph Accessed: 2026-02-23.

[18] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., Red Hook, NY, USA, 9459–9474. https://arxiv.org/abs/2005.11401

[19] Llama Stack Contributors. 2025. Llama Stack. GitHub repository. https://github.com/llamastack/llama-stack Accessed: 2026-01-28.

[20] Llama Stack Contributors. 2025. Llama Stack Kubernetes Operator. GitHub repository. https://github.com/llamastack/llama-stack-k8s-operator Accessed: 2026-01-28.

[21] LlamaIndex. 2022. LlamaIndex: Data framework for LLM applications. GitHub repository. https://github.com/run-llama/llama_index Accessed: 2026-02-24.

[22] Microsoft. 2023. AutoGen: A programming framework for agentic AI. GitHub repository. https://github.com/microsoft/autogen Accessed: 2026-02-24.

[23] Microsoft. 2023. Semantic Kernel: Integrate cutting-edge LLM technology quickly and easily into your apps. GitHub repository. https://github.com/microsoft/semantic-kernel Accessed: 2026-02-24.

[24] Microsoft. 2025. Microsoft Agent Framework. GitHub repository. https://github.com/microsoft/agents Accessed: 2026-02-24.

[25] OGX Contributors. 2026. OGX Kubernetes Operator. GitHub repository. https://github.com/ogx-ai/ogx-k8s-operator Formerly Llama Stack Kubernetes Operator. Accessed: 2026-04-25.

[26] OGX Contributors. 2026. OGX (Open GenAI Stack). GitHub repository. https://github.com/ogx-ai/ogx Formerly Llama Stack. Accessed: 2026-04-25.

[27] Open Responses Community. 2026. Open Responses. Online resource. https://www.openresponses.org/ Accessed: 2026-02-23.

[28] OpenAI. 2023. Function Calling. Online documentation. https://platform.openai.com/docs/guides/function-calling Accessed: 2026-02-24.

[29] OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925 [cs.CL] https://arxiv.org/abs/2508.10925

[30] OpenAI. 2025. Responses API Reference. Online documentation. https://platform.openai.com/docs/api-reference/responses Accessed: 2026-02-16.

[31] Pydantic. 2024. Pydantic AI: Agent Framework / shim to use Pydantic with LLMs. GitHub repository. https://github.com/pydantic/pydantic-ai Accessed: 2026-02-24.

[32] Kunal Sawarkar, Abhilasha Mangal, and Shivam Raj Solanki. 2024. Blended RAG: Improving RAG (Retriever-Augmented Generation) Accuracy with Semantic Search and Hybrid Query-Based Retrievers. *arXiv preprint arXiv:2404.07220* 1, 1 (2024), 1–12. https://arxiv.org/abs/2404.07220

[33] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., New Orleans, LA, USA, 1–25. https://arxiv.org/abs/2302.04761

[34] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. 2015. Hidden Technical Debt in Machine Learning Systems. In *Advances in Neural Information Processing Systems*, Vol. 28. Curran Associates, Inc., Red Hook, NY, USA, 2503–2511. https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems

[35] Natalie Shapira, Chris Wendler, Avery Yen, Gabriele Sarti, Koyena Pal, Olivia Floody, Adam Belfki, Alex Loftus, Aditya Ratan Jannali, Nikhil Prakash, Jasmine Cui, Giordano Rogers, Jannik Brinkmann, Can Rager, Amir Zur, Michael Ripa, et al. 2026. Agents of Chaos. *arXiv preprint arXiv:2602.20021* 1, 1 (2026), 1–25. https://arxiv.org/abs/2602.20021

[36] Vectara. 2023. Vectara: Enterprise Agent and RAG Platform. Online. https://vectara.com/ Accessed: 2026-02-24.

[37] Weaviate. 2019. Weaviate: Cloud-native vector database with structured filtering. GitHub repository. https://github.com/weaviate/weaviate Accessed: 2026-02-24.

[38] Simon Willison. 2022. Prompt injection attacks against GPT-3. Blog post. https://simonwillison.net/2022/Sep/12/prompt-injection/ Accessed: 2026-02-24.

[39] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*. OpenReview, Kigali, Rwanda, 1–18. https://arxiv.org/abs/2210.03629

[40] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, Carlsbad, CA, USA, 521–538. https://www.usenix.org/conference/osdi22/presentation/yu

[41] Matei Zaharia, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Konwinski, Siddharth Murching, Tomas Nykodym, Paul Ogilvie, Mani Parkhe, Fen Xie, and Corey Zumar. 2018. Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Engineering Bulletin* 41, 4 (2018), 39–45. https://people.eecs.berkeley.edu/~matei/papers/2018/ieee_mlflow.pdf

[42] Zilliz. 2019. Milvus: A cloud-native vector database. GitHub repository. https://github.com/milvus-io/milvus Accessed: 2026-02-24.

A Detailed Evaluation Tables

Config	Orch. / Retr.	p50	p99	Mean
A	Client / Ungated	3,600ms	10,818ms	4,208ms
B	Client / Gated	3,427ms	9,795ms	3,851ms
C	Server / Ungated	7,507ms	16,462ms	7,620ms
D	Server / Gated	6,431ms	14,623ms	6,934ms

Table 3: End-to-end latency for authorized queries. Total latency variation between gated and ungated is dominated by external API response times. Server-side orchestration adds ∼3s due to the Responses API tool execution round-trip (non-streaming).

Component	Median	P95	N
vLLM Direct (baseline)	447.9ms	531.5ms	50
OGX (routing + dispatch)	452.6ms	537.9ms	50
Search (ungated)	283.9ms	294.3ms	50
Search (tenant-gated)	289.4ms	306.0ms	50

Table 4: GPU infrastructure overhead (vLLM on T4, no auth). Routing adds 4.7ms (1.0%); metadata filtering adds 5.5ms (1.9%). With auth enabled, an additional ∼14ms brings total overhead to ∼19ms.

Config	Orch. / Retr.	c=1	c=5	c=10	c=25
A	Client / Ungated	0.5	1.6	2.2	5.4
B	Client / Gated	0.5	1.5	2.2	4.2
C	Server / Ungated	0.2	0.8	0.8	2.2
D	Server / Gated	0.2	0.9	1.5	2.6

Table 5: Throughput (QPS) at four concurrency levels. Gating does not degrade throughput. Client-side orchestration achieves ∼2× QPS at high concurrency.

Config	Orch. / Retr.	Probes	Leaked	Leak Rate
A	Client / Ungated	90	72	80.0%
B	Client / Gated	90	0	0.0%
C	Server / Ungated	90	56	62.2%
D	Server / Gated	90	0	0.0%

Table 6: Prompt injection probe results. Gated configs block all probes; the defense is at the retrieval layer, not the model.

Corpus Size	Gated Latency	Filter Overhead	Recall@5
100	3.79ms	0.74ms	1.000
1,000	3.93ms	0.79ms	0.100
10,000	5.21ms	1.00ms	0.010
50,000	11.43ms	2.95ms	0.002

Table 7: Post-retrieval filtering scaling at 5× over-fetch (sqlite-vec). Latency overhead is small regardless of corpus size; recall degrades as cross-tenant documents contaminate the top-k set. Pushdown-capable backends maintain Recall@5=1.000 at all sizes.

Configuration	Recall@5	Precision@5	MRR
Ungated	1.000	0.200	0.700
Chunk-level gated	1.000	0.433	1.000
Per-tenant index	1.000	0.200	1.000

Table 8: Retrieval quality with synthetic embeddings (∼0.95 cross-tenant similarity). Gating improves precision by 2.2× by filtering cross-tenant noise.

B API and Provider Details

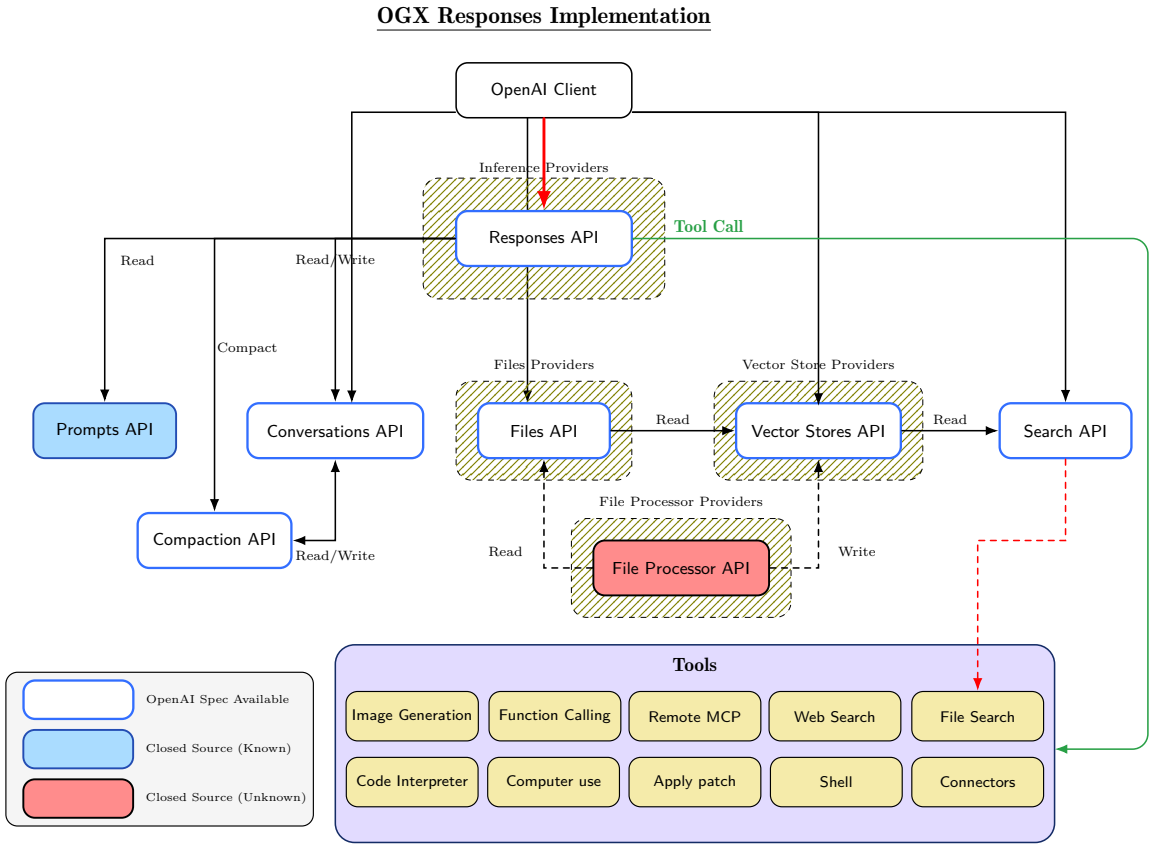

Figure 5: Responses API surface area and its relation to other APIs, providers, and tools. The Responses API serves as the central orchestration endpoint, coordinating inference, tool execution, state management, and context compaction.

Figure 5 illustrates the complete Responses API surface area and the interconnections between APIs, providers, and tools. OGX implements the Responses API not only as an agentic execution endpoint but as a resource model encompassing vector stores, files, and conversations—each a first-class, tenant-scoped API object subject to the same ABAC policies. The key APIs include:

- **Responses API:** Central orchestration endpoint for multi-turn conversations, tool calling, and agentic execution [30].
- **Vector Stores and Search APIs:** Dense, sparse, and hybrid retrieval with structured metadata filtering for tenant isolation.
- **Conversations API:** Multi-turn conversation state with tenant-scoped isolation.
- **Compaction API:** Context management for long conversations. Summarizes history via inference when token count exceeds a threshold, preserving user messages verbatim. Supports explicit (`POST /v1/responses/compact`) and automatic modes.
- **Prompts API:** Versioned prompt template management with tenant-scoped access control.
- **Files and File Processors APIs:** Tenant-scoped file storage (GCS, S3, PVCs, local) with parsing and chunking for vector store ingestion.
- **Tools:** Built-in tools (file search, web search, code interpreter, image generation, computer use) and external tools via MCP [2].

Supported inference providers include vLLM [15], Ollama, OpenAI, Anthropic, Azure, AWS Bedrock, Databricks, Gemini, Together, NVIDIA, and WatsonX. Vector store providers include Chroma, pgvector, Elasticsearch, Qdrant, Weaviate, Milvus, Oracle Cloud Infrastructure, FAISS [12], and sqlite-vec. The Kubernetes Operator [25] enables deployment of heterogeneous backends as shared services, with multiple OGX instances referencing the same providers while maintaining logical isolation through ABAC.